Databases on AWS: Aurora, RDS and DynamoDB

Relational vs non-relational, how Aurora works, and how to choose

1. Two families of databases

Relational (SQL) databases store data in **tables** with rows and columns. Tables are linked by **keys**, and you ask questions in **SQL**. The schema (structure) is fixed in advance. **Non-relational (NoSQL)** databases store data in other shapes (key-value, document, graph, wide-column), usually with a flexible schema, and are built to scale out across many machines.

	Relational (SQL)	Non-relational (NoSQL)
Data shape	Tables, rows, columns	Key-value, documents (JSON), graph, wide-column
Schema	Fixed and enforced	Flexible, can differ per item
Relationships	JOINs across tables, foreign keys	Usually denormalised: related data stored together
Queries	Powerful ad-hoc SQL (filters, joins, aggregates)	Fast for the access patterns you designed for; limited ad-hoc queries
Consistency	**ACID** transactions	Often tunable; many offer eventual consistency (some support transactions)
Scaling	Mostly vertical (bigger instance) + read replicas	Horizontal (more nodes), very large scale
AWS services	RDS, **Aurora**, Redshift	**DynamoDB**, DocumentDB, ElastiCache, Neptune, Keyspaces

ACID in one line each: **A**tomic (all steps of a transaction succeed or none do), **C**onsistent (rules and constraints always hold), **I**solated (transactions do not disturb each other), **D**urable (committed data survives a crash). Bank transfers and clinical records need ACID.

2. When to use which

Choose RELATIONAL when...	Choose NON-RELATIONAL when...
Data has clear structure and many relationships (customers, orders, subjects, visits)	Access is simple: get or put an item by key, at huge scale
You need complex queries, joins and reports	You need single-digit-millisecond responses at any scale
You need strong transactions (money, medical or regulated records)	Data shape changes often, or varies from item to item
Data integrity rules must be enforced by the database	You have massive write volume (IoT, clickstream, gaming, sessions)
Your team already knows SQL	You want a serverless database with no capacity to manage

Modern reality: most systems use **both**. For example, a shop keeps orders and payments in Aurora (relational), the shopping cart and sessions in DynamoDB (key-value), hot product pages in a cache (ElastiCache) and old order history in S3, queried with Athena. This is called **polyglot persistence**: the right store for each job.

3. Amazon RDS (Relational Database Service)

RDS is a **managed** relational database. AWS handles the server, operating system, patching, backups and failover. You choose the engine and size. You do **not** get SSH access to the server, which is the price of it being managed.

- **Engines:** Aurora (MySQL- and PostgreSQL-compatible), MySQL, PostgreSQL, MariaDB, Oracle, SQL Server (and Db2).
- **Automated backups:** daily snapshot plus transaction logs; retention up to 35 days; restore to any second in that window (into a **new** instance).
- **Manual snapshots:** kept until you delete them.
- **Storage:** EBS-based, with autoscaling you can enable. **Encryption** is chosen at creation (KMS).
- **Parameter groups** hold engine settings; **subnet groups** say which subnets (at least 2 AZs) the database may use.

Feature	Purpose	Key point
Multi-AZ	**High availability**: a standby copy in another AZ, with automatic failover	In the classic setup the standby is **not readable**. It is for survival, not for speed. (A newer Multi-AZ *cluster* option for MySQL/PostgreSQL has readable standbys.)
Read replica	**Performance**: extra copies for read traffic	Replication is **asynchronous** (small lag). Can be in another Region. Can be promoted to a standalone database.
RDS Proxy	Pools and reuses database connections	Very useful for **Lambda**, which can open thousands of short connections

Classic exam trap: Multi-AZ is for **availability**, read replicas are for **scaling reads**. They solve different problems and can be used together.

4. Amazon Aurora

Aurora is AWS's own relational engine, **compatible with MySQL and PostgreSQL** (applications and drivers for those work with little or no change). It is part of RDS, but rebuilt for the cloud. Its big idea: **separate compute from storage**.

Topic	How Aurora does it
Storage	One shared cluster volume, **6 copies across 3 AZs**, self-healing, grows automatically in 10 GB steps up to 128 TiB. No disk size to choose.
Instances	One **writer** plus up to **15 readers**, all using the same storage, so reader lag is typically milliseconds.
Endpoints	**Cluster endpoint** = always the current writer. **Reader endpoint** = load-balanced across readers (per connection). Never hard-code an *instance* endpoint in an app.
Failover	If the writer fails, a reader is promoted, usually in well under a minute. The cluster endpoint then points at the new writer.
Backups	Continuous, retention 1 to 35 days, restore creates a **new cluster** with a new endpoint.
Cloning	Copy a large database in minutes with copy-on-write, great for testing a risky change.
Backtrack	Rewind in place without restoring (Aurora MySQL only).
Serverless v2	Capacity scales automatically (in ACUs) with load. Good for variable or unpredictable workloads. Check current docs for minimum capacity and options.
Global Database	Replicates to other Regions, typically under 1 second, for disaster recovery and fast local reads worldwide.

Topic	How Aurora does it
Security	Encryption at rest (chosen at creation), TLS in transit, private subnets, security group allowing 3306 (MySQL) or 5432 (PostgreSQL) only from the app tier, IAM database authentication, **Secrets Manager** for credentials.

RDS (standard engines) vs Aurora

	RDS MySQL / PostgreSQL	Aurora
Storage	EBS volume per instance	Shared, 6 copies in 3 AZs
Read scaling	Fewer replicas, each with its own copy	Up to 15 readers sharing storage
Failover	Standby promoted, often 1 to 2 minutes	Reader promoted, usually under a minute
Cost	Lower per instance hour	Higher per hour, extra options; **generally not Free Tier**
Pick it when	Small or simple app, tight budget, an engine Aurora lacks (Oracle, SQL Server)	Production apps needing higher availability, many readers, fast failover

Cost warning: an Aurora cluster bills every hour it exists, even idle. If you create one to practise, **delete it the same day** (and remove snapshots you don't need). Check the current Aurora pricing page before building.

5. Amazon DynamoDB (the main NoSQL service)

DynamoDB is a **fully managed, serverless key-value and document** database with single-digit-millisecond performance at almost any scale. There are no servers, patches or disks to manage.

- **Table → items → attributes.** An item is like a row but each item can have different attributes. Maximum item size is 400 KB.
- **Primary key:** a **partition key** alone, or a **partition key + sort key**. The partition key decides where the data physically lives, so choose one with **many distinct values** (such as user_id) to spread load evenly. A key like "status" with only two values creates a hot partition.
- **You design the table around your queries** (access patterns), the opposite of relational design. Joins do not exist.
- **Capacity modes: on-demand** (pay per request, no planning, good for spiky or new workloads) and **provisioned** (set read/write capacity, optionally with auto scaling, cheaper for steady load).
- **Indexes:** Global Secondary Index (GSI) lets you query by a different key; Local Secondary Index (LSI) uses the same partition key with a different sort key.
- **Extras:** TTL (auto-expire items), Streams (react to changes, for example with Lambda), point-in-time recovery, **global tables** (multi-Region), **DAX** (in-memory cache), transactions.
- **Consistency:** reads are eventually consistent by default; you can request strongly consistent reads (for a single Region).

Real-world example: DynamoDB for a shopping cart
Table **Carts**, partition key **user_id**, sort key **item_id**. Reading a user's whole cart is one fast query on user_id. TTL deletes abandoned carts after 30 days. No joins needed, and it handles a sale-day spike without any capacity planning in on-demand mode.

6. Other AWS databases (know the one-line purpose)

Service	Type	Use it for
Redshift	Data warehouse (relational, columnar)	Analytics (OLAP) over very large data. Not for day-to-day app transactions.
ElastiCache	In-memory cache (Redis / Memcached)	Sub-millisecond reads, sessions, leaderboards; takes load off the main database
DocumentDB	Document (MongoDB-compatible)	Existing MongoDB-style applications
Neptune	Graph	Relationships: social networks, fraud detection, knowledge graphs
Keyspaces	Wide-column (Cassandra-compatible)	Existing Cassandra workloads
Timestream	Time series	IoT and monitoring metrics
Athena + S3	Serverless SQL on files	Query CSV/Parquet in S3 without a database
DMS	Migration service	Move or replicate databases (also between engines, with the Schema Conversion Tool)

OLTP vs OLAP: OLTP is many small transactions (an app writing orders): use RDS or Aurora. OLAP is a few huge analytical queries (reports over years of data): use Redshift or Athena.

7. The decision table

Need	Choose
A standard relational app database, small or simple	RDS (MySQL or PostgreSQL)
Relational, production, highest availability, many readers	**Aurora**
Relational but with unpredictable or spiky load	Aurora Serverless v2
Oracle or SQL Server licence	RDS for Oracle or SQL Server
Key-value at huge scale, serverless, millisecond latency	**DynamoDB**
Cache to speed up a slow database	ElastiCache
Large analytics and reporting	Redshift (or Athena on S3)
Data with highly connected relationships	Neptune
MongoDB application	DocumentDB

8. Good practice (what real teams do)

- Put databases in **private subnets**; never make them publicly accessible.
- Security group: allow the database port **only from the application's security group**.
- Store credentials in **Secrets Manager**, never in code or in a Git repo.
- Encrypt at rest (KMS) and in transit (TLS). Choose encryption **when you create** the database.
- Turn on automated backups, and test a restore at least once. A backup you never restored is only a hope.
- Use **Multi-AZ** or Aurora replicas for production, and monitor with CloudWatch (CPU, connections, replica lag, free storage).
- Add **indexes** for columns you filter or join on; use `EXPLAIN` to see how a query runs.

- Use RDS Proxy when many short-lived clients (like Lambda) connect.
- Set billing alarms and delete practice databases.

Real-world example: a clinical study database
Subjects, visits and adverse events have strict relationships (every visit belongs to a subject; a severity must be MILD, MODERATE or SEVERE) and reviewers need joins and summaries by treatment arm. That is a **relational** job. A team would use **Aurora PostgreSQL or MySQL** in private subnets, encrypted, with backups and an audit trail (CloudTrail plus database logs). Raw files from the data-capture system stay in **S3**, and device or app event streams could go to **DynamoDB**. Regulated environments also need formal validation, which is a process and not just a service choice.

9. Hands-on practice: SQL on your own laptop (free)

You don't need a paid database to learn the SQL that Aurora uses. This repo's `sql/` folder contains a small synthetic clinical-trial database (sites, subjects, visits, adverse events) and 7 practice queries (GROUP BY, JOIN, LEFT JOIN, HAVING and a window function).

```
# Option A: SQLite (free, nothing to install except Python)
python sql/run_sqlite.py

# Option B: MySQL / Aurora MySQL
mysql -h <endpoint> -u admin -p < sql/01_schema.sql
mysql -h <endpoint> -u admin -p < sql/02_data.sql
mysql -h <endpoint> -u admin -p < sql/03_queries.sql
```

- Run each query and **guess the result before you run it**. That is how you learn SQL fast.
- Query 5 uses a LEFT JOIN to find subjects with no adverse events. Change it to an INNER JOIN and see who disappears.
- Uncomment the age-12 insert (query 8) and see the database refuse it. That is a constraint protecting data quality.
- Aurora lab (optional, costs money): create a small Aurora MySQL cluster, load these files from an EC2 instance, then compare the cluster and reader endpoints. The steps are in the earlier Route 53 + Aurora notes. **Delete the cluster when finished.**

10. Interview questions

Question	Short answer
Relational vs NoSQL?	Relational: structured tables, joins, ACID. NoSQL: flexible schema and horizontal scale, designed around access patterns.
Multi-AZ vs read replica?	Multi-AZ is for availability (failover). A read replica is for read performance (async copy).
What makes Aurora different from RDS MySQL?	Shared storage with 6 copies across 3 AZs, up to 15 readers on the same storage, faster failover.
Which Aurora endpoint do apps use for writes?	The cluster (writer) endpoint, which always points at the current writer.
How do you choose a DynamoDB partition key?	High cardinality so load spreads evenly, matching how you query.
On-demand vs provisioned capacity?	On-demand: pay per request, no planning. Provisioned: set capacity, cheaper for steady workloads.
Why RDS Proxy with Lambda?	It pools connections, so thousands of short Lambda calls do not exhaust the database.
OLTP vs OLAP?	OLTP: many small transactions (RDS, Aurora). OLAP: big analytical queries (Redshift, Athena).
How do you secure a database on AWS?	Private subnet, security group from the app tier only, encryption, Secrets Manager, IAM, backups, logging.

Quick quiz

1 Your app needs complex reports with joins across five tables. SQL or NoSQL?

2 Which setting keeps your database running if an entire AZ fails, and is the standby readable in the classic RDS setup?

3 How many copies of data does Aurora keep, and across how many AZs?

4 You want to send read-only dashboard traffic away from the writer. What do you use?

5 A DynamoDB table uses "country" (only 5 values) as its partition key. What is wrong?

6 Where should the database password be stored?

7 Which service is built for large analytical queries?

Answers: 1) SQL (relational). 2) Multi-AZ; no, the classic standby is not readable. 3) 6 copies across 3 AZs. 4) The Aurora reader endpoint (or an RDS read replica). 5) Too few distinct values: a hot partition and uneven load. 6) AWS Secrets Manager (not in code). 7) Redshift (or Athena on S3).

Prices, limits and feature details change. Check the current AWS documentation and pricing pages before relying on any number here.